

Explicit Architecture, with a Go accent

A detailed map of Mixology: conceptual responsibility, supported dependencies, execution, and evidence.

DRIVING SIDE

CLI · main/cli
urfave/cli flags + command hierarchy
Parse args / JSON; print results
Fresh invocation context

TUI · main/tui
Bubble Tea messages + commands
ViewModel / Interaction ownership
Forms, dialogs, keys, text frames

GUI · main/gui
Fyne presenters + retained widgets
Executor → UI dispatcher
Live input and request generations

Independent adapter mechanics
surfaces/{cli,tui,gui} per domain
Matching pkg/toolkits only
main/seed supplies sample data

PUBLIC APPLICATION OPERATIONS



DRIVEN TOOLS + OWNED ADAPTERS

pkg/store → SQLite
Owned Store / Tx / Query[T]
Private DAO rows → JSON records
Registration → indexes + constraints
WAL readers; serialized writers
Revision predicate rejects stale writes

Domain authz → Cedar
Policies and schemas owned per context
Input and result authorization
Public models expose Cedar entities
Generated policy assembly in pkg/authz

Typed filtering → Expr + SQL
Domain schema → checked expression
Safe SQL candidate constraints
Hydration → complete predicate
Authorized paging yields visible matches

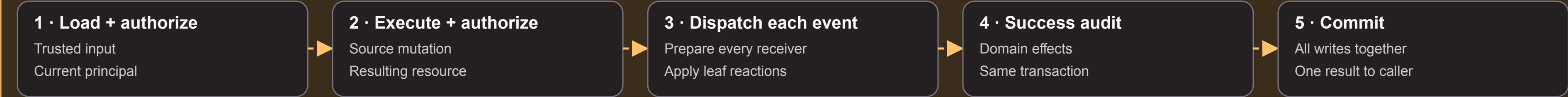
Store changes → client refresh
Connection-local data_version monitor
Coalesced invalidation; no record payload
TUI / GUI repeat authorized queries
Separate from transactional domain events

Circle = responsibility map, not a literal import graph. Public contracts remain coupled to their named owners.

WITHIN A REGULAR CONTEXT / OWNERSHIP AND VISIBILITY

surfaces → public facade → typed pipeline → internal/commands + internal/dao; queries and handlers may use their own DAO.
models / queries / events are public contracts. Tagging reacts through handlers and owns association writes in internal/dao.
Peer reads use supported query contracts. Commands publish only their own events. Handlers cannot import facades or commands.

EXECUTION / ONE LOCAL TRANSACTION / NUMBERED DASHED ARROWS



Retirement: IngredientDeleted → Drinks / Inventory / Menus / Orders. Preparation is per event; HandlerContext cannot AddEvent.
Failure: owner rolls back, then records failed attempt. One command claims each transaction; nested and sequential command composition is rejected.

SHARED VALUES + NARROW PORTS
app/kernel: typed entity IDs, money, measurement, quality, tags, tag.Repository.
Common identity and unit semantics.
Tagging owns associations; registered domain loaders preserve target ownership.
Kernel does not depend on domains.

MECHANISMS + EXPLICIT WIRING
middleware: transactions, activity, dispatch.
app.New injects dispatcher + audit writer.
pkg/dispatcher imports domain receivers.
pkg/authz assembles domain policy material.
Generators: events, policies, IDs, errors.
pkg is not uniformly domain-independent.

OBSERVATION + PRESENTATION
Gets authorize results; lists elide denials.
Action state: hidden / disabled / enabled.
Commands repeat policy and prerequisites.
Errors keep typed, transport-neutral meaning.
Log + metrics surround operation paths.
Audit explains effects; it is not a replay log.

EXECUTABLE EVIDENCE

Types + Go internal	Captured import rules	Topology + registration	Permutation + rollback	Independent clients
Current scope: seven contexts and three UI runtimes. Procurement, Analytics, process managers, brokers, and outboxes are future design pressures.				

Selected dependency / contract use Execution sequence (not imports) Colored regions group responsibilities; linked panels provide the code trace.